

ROADMAP DE CARRIÈRE

Software Engineer → Middle East

OBJECTIF FINAL

Devenir une Software Engineer solide en Java/Spring Boot, capable de travailler sur des applications modernes, cloud et distribuées, puis décrocher une opportunité dans le Middle East (Saudi Arabia / Qatar / UAE).

1. ■ FORMATION

- Chercher un Cycle Ingénieur en Génie Logiciel
- Vérifier la reconnaissance et l'accréditation du diplôme
- Vérifier la valeur du diplôme à l'international
- Vérifier les stages / alternance et partenariats entreprises
- Réussir mon cycle ingénieur
- Ne pas choisir une filière uniquement parce qu'elle contient « AI » ou « Cyber »

2. ■ JAVA — DEVENIR TRÈS FORTE

Java Core : variables/types, conditions, boucles, méthodes, classes/objets, encapsulation, héritage, polymorphisme, interfaces, classes abstraites, exceptions, collections, List/Set/Map, HashMap, equals/hashCode, Generics, Lambda, Streams, Optional, Records, Enums.

Java avancé : multithreading, concurrency, Threads/Executors, CompletableFuture, JVM basics, Memory/Garbage Collector, Java 21+.

- Pouvoir réussir un entretien technique Java sans paniquer

3. ■ ALGORITHMES & LOGIQUE

- Big O ■ Arrays ■ Strings ■ HashMap/HashSet ■ Stack ■ Queue ■ Linked List
- Binary Search ■ Sorting ■ Recursion ■ Two Pointers ■ Sliding Window
- Trees — bases ■ Graphs — bases
- Résoudre 3–5 problèmes d'algorithmique par semaine

4. ■ SPRING BOOT — DOMAINE PRINCIPAL

- Spring Boot ■ Controller/Service/Repository ■ REST API ■ DTO ■ Entity
- Validation ■ Exception handling ■ GlobalExceptionHandler
- JPA ■ Hibernate ■ Relationships (OneToOne, OneToMany, ManyToOne, ManyToMany)
- JPQL ■ Specifications ■ Pagination ■ Sorting ■ Filtering ■ Transactions
- Spring Profiles ■ Configuration ■ Environment variables ■ Swagger/OpenAPI

5. ■ SPRING SECURITY

- Authentication ■ Authorization ■ JWT ■ Access Token ■ Refresh Token
- Password hashing ■ BCrypt ■ Roles ■ Permissions ■ RBAC
- SecurityFilterChain ■ Filters ■ UserDetails ■ UserDetailsService
- CORS ■ CSRF ■ Method Security ■ @PreAuthorize
- Être capable d'expliquer le parcours d'une requête et la décision d'autorisation

6. ■■ DATABASE

SQL : SELECT, INSERT, UPDATE, DELETE, JOIN, GROUP BY, HAVING, ORDER BY, subqueries, indexes, constraints, transactions, normalization, ACID.

■ PostgreSQL — base principale ■ MySQL ■ Database design ■ Query optimization

7. ■■ REACT

■ Components ■ JSX ■ Props ■ State ■ useState ■ useEffect ■ Rendering ■ DOM
■ Events ■ Forms ■ useForm ■ Context API ■ Custom hooks
■ React Router ■ useNavigate ■ Axios ■ Interceptors
■ Authentication ■ Protected routes ■ Error handling ■ Loading states ■ MUI
■ Créer un frontend React proprement connecté à une API Spring Boot

8. ■ DOCKER

■ Image ■ Container ■ Dockerfile ■ Docker Compose ■ Volumes ■ Networks
■ Environment variables ■ Ports ■ Docker Hub
■ Containeriser Spring Boot, React et PostgreSQL
■ Faire tourner React + Spring Boot + PostgreSQL + Redis avec Docker Compose

9. ■ GIT & GITHUB

■ init ■ clone ■ add ■ commit ■ push ■ pull ■ branch ■ merge ■ rebase — bases
■ Pull Request ■ Resolve conflicts ■ .gitignore ■ GitHub Actions
■ README professionnel ■ GitHub profile propre

10. ■ CI/CD

■ Comprendre CI/CD ■ GitHub Actions ■ Build automatique ■ Tests automatiques
■ Docker build ■ Docker image ■ Deployment automatique

11. ■■ CLOUD — TRÈS IMPORTANT

Choisir AWS : IAM, EC2, S3, RDS, VPC, Security Groups, Load Balancer, CloudWatch.

■ Déployer Spring Boot ■ Déployer React ■ PostgreSQL sur AWS ■ Docker sur AWS
■ AWS Certified Cloud Practitioner
■ Puis éventuellement AWS Certified Developer – Associate

12. ■■ KUBERNETES

■ Concepts Kubernetes ■ Pod ■ Deployment ■ Service ■ ConfigMap ■ Secret
■ Ingress ■ Namespace ■ Scaling ■ Kubernetes deployment

13. ■ MICROSERVICES

■ Monolith ■ Microservices ■ API Gateway ■ Service Discovery ■ OpenFeign
■ Communication entre services ■ Database per service
■ Distributed transactions — bases ■ Resilience ■ Circuit Breaker ■ Retry
■ Kafka ■ Event-driven architecture

14. ■ REDIS

- Pourquoi Redis ■ Cache ■ Key/Value ■ TTL ■ Cache Spring Boot ■ Session/Cache
- Rate limiting — bases

15. ■ KAFKA

- Producer ■ Consumer ■ Topic ■ Partition ■ Offset ■ Consumer Group
- Event-driven architecture ■ Kafka avec Spring Boot

16. ■ MONITORING & OBSERVABILITY

- Spring Boot Actuator ■ Metrics ■ Prometheus ■ Grafana
- Logs ■ Loki ■ Alertmanager ■ Health checks
- Comprendre comment détecter une panne avant que l'utilisateur ne la signale

17. ■ TESTING

- JUnit 5 ■ Mockito ■ Unit tests ■ Integration tests
- Controller tests ■ Service tests ■ Repository tests ■ Mocking ■ Test coverage
- Testcontainers — plus tard
- Ajouter des tests à chaque gros projet

18. ■■ SOFTWARE ENGINEERING

- Clean Code ■ SOLID ■ Design Patterns ■ MVC ■ Layered Architecture
- Clean Architecture — bases ■ Dependency Injection ■ DRY ■ KISS
- Separation of Concerns ■ UML ■ System Design ■ API Design
- Scalability ■ Security

19. ■ AI ENGINEERING

- Python basics ■ NumPy/Pandas — bases ■ Machine Learning — concepts
- LLM ■ Prompt engineering ■ AI APIs ■ Embeddings ■ Vector databases
- RAG ■ Function calling / tool use ■ AI agents — bases ■ Spring AI
- Intégrer l'IA dans une application Spring Boot

20. ■■ ANGLAIS

- Atteindre B2 solide puis C1
- Parler anglais chaque semaine ■ Lire la documentation technique en anglais
- Faire des entretiens techniques en anglais
- Présenter mes projets en anglais ■ Expliquer Spring Security en anglais
- Expliquer mon architecture en anglais
- Pouvoir passer un entretien technique complet en anglais

21. ■ EXPÉRIENCE PROFESSIONNELLE

- Trouver un stage/alternance sérieux ■ Obtenir une première expérience CDI
- Atteindre 1 an d'expérience ■ Puis 2 ans d'expérience
- Travailler sur de vrais systèmes ■ Participer aux code reviews
- Travailler avec Git/Jira ■ Participer aux réunions Agile/Scrum
- Écrire des tests ■ Déployer des applications

22. ■ MES PROJETS

Projet 1 — LogiTrack : améliorer Microservices, Security, Docker, CI/CD, AWS, Redis, Kafka, Monitoring, Tests, README, Architecture diagram.

Projet 2 — SaaS : React + Spring Boot + PostgreSQL + Redis + Docker + AWS, avec authentication, subscription, roles, dashboard, emails et monitoring.

Projet 3 — AI : Spring Boot + React + AI + RAG pour une vraie application.

■ Éviter de multiplier les petits CRUD ; privilégier 3 projets solides et présentables.

23. ■■ MIDDLE EAST

■ LinkedIn en anglais ■ CV en anglais ■ GitHub professionnel ■ Portfolio

■ Préparer les candidatures Saudi Arabia ■ Qatar ■ UAE

■ Rechercher les entreprises tech ■ Suivre les recruteurs ■ Participer aux événements tech

■ Postuler régulièrement ■ Préparer les entretiens techniques

■ ORDRE D'APPRENTISSAGE

1. ■ Cycle Ingénieur → 2. ■ Java → 3. ■ Algorithmique/logique → 4. ■ Spring Boot → 5. ■ Spring Security → 6. ■■ PostgreSQL/SQL → 7. ■■ React → 8. ■ Docker → 9. ■ Testing → 10. ■ CI/CD → 11. ■■ AWS → 12. ■ Microservices → 13. ■ Redis + Kafka → 14. ■■ Kubernetes → 15. ■ Observability → 16. ■ AI Engineering → 17. ■■ Middle East

■ RÈGLE PRINCIPALE

Ne cherche pas à connaître tout en même temps. Ton premier objectif est de devenir très forte en **Java + Spring Boot + SQL + React + Git + Docker**. Ensuite, ajoute progressivement Cloud, Microservices, DevOps et AI.

■ MON OBJECTIF FINAL

« Je ne veux pas seulement obtenir un diplôme. Je veux devenir une Software Engineer capable de concevoir, développer, tester, sécuriser, dockeriser et déployer une application professionnelle de A à Z, puis utiliser cette expertise pour construire une carrière internationale dans le Middle East. »